Designing a Fibonacci Accelerator

Authors:
WhisperingRock

Class
CPE490: Open-Source ASIC Design
November 1, 2026

Contents

Contents . . . 1

1 TODOs **2**

2 Introductions **3**

3 Pre-Synthesis **4**
3.1 Procedure . . . 4
3.2 Design . . . 4
3.3 Simulation Testing . . . 10
3.3.1 Brent-Kung 'Parallel Prefix' Adder . . . 10
3.3.2 Fibonacci Accelerator . . . 11

4 Initial Post-Synthesis Analysis **12**
4.1 Static Timing Analysis (Pre-PlaceNRoute) . . . 12
4.1.1 Analysis . . . 13
4.1.2 Redesign . . . 13

5 Appendix **14**
5.1 Code . . . 14
5.1.1 RTL . . . 14
5.1.2 Simulation SVA Testing . . . 18
5.2 Boneyard . . . 20

6 References **21**

Chapter 1

TODOs

1. Formal verif BMC of fib, not just corner
2. STA documentation
3. STA digestion
4. STA potential fixes

Chapter 2

Introductions

In this lab, the task was to create a Fibonacci accelerator. Before going further in the paper, I must first describe the Fibonacci sequence and what makes a design an *accelerator*.

First, let's discuss the Fibonacci sequence using a recursive formula.

$$\boldsymbol{F_{n+2} = F_{n+1} + F_n} \text{ where } \boldsymbol{F_0 = 0} \text{ and } \boldsymbol{F_1 = 1}$$

In our discussion in class, another relationship emerged when comparing multiple terms.

Consider:

- Let $F_n = a$
- Let $F_{n+1} = b$
- $F_{n+2} = a + b$
- $F_{n+3} = a + 2b$
- $F_{n+4} = 2a + 3b$
- $F_{n+5} = 3a + 5b$

If given two terms of the sequence, we can potentially calculate the next four terms from the initial two. This unique way to describe terms has potential for a speed-up if we can utilize shifters and adders with the motivation to calculate many terms in a single cycle.

This motivation to increase throughput or performance over a CPU, through the use of configurable logic, is termed as an *accelerator*. For this lab, our accelerator will calculate more than one term of the Fibonacci sequence per clock cycle.

Lastly, another motivation of this lab is to become familiar with open-source tooling set.

Chapter 3

Pre-Synthesis

3.1 Procedure

The workflow of this lab, which is largely focused on design before synthesis, will be the following

1. LINTing based off of many tools s.a. Verilator, Icarus, GL, ...
2. Simulation of test scripts on Verilator, then Icarus Verilog
3. BMC verification
4. K-Induction formal verification (optional)

3.2 Design

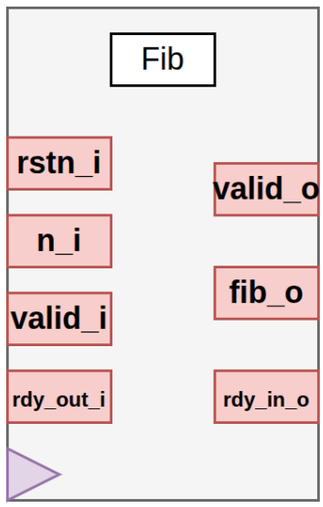


Figure 3.1: Fibonacci Top Module

The top-level module has a data and control signals:

- **clk_i**: clock (posedge is captured).
- **rstn_i**: active-low synchronous reset.
- **valid_i**: indicates when input n_i can be read.
- **n_i**: requested term of fib sequence.
- **rdy_out_i**: indicates when downstream has read this modules output (and module can begin processing a new sequence).

- **valid_o**: indicates module's output is ready to be read.
- **rdy_on_o**: indicates module is ready for a new request (n_i).
- **fib_o**: modules output (F_n).

In order to capture the functionality of these top-level signals, an FSM was used:

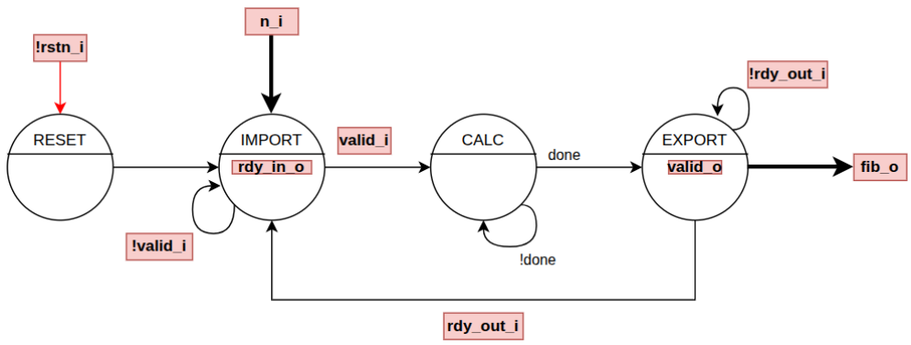


Figure 3.2: FSM of Fibonacci Accelerator

From here, lets take a closer look into the CALC stage of the FSM, where most of our accelerator logic comes into play. An attentive reader will notice that there are only two adders and a single shifter: there is a considerable amount of adders required to further calculate F_{n+4} and F_{n+5}. Since this must be implemented in ASIC where wires and hardware are not free lunches, this designer opted to take 2 cycles to calculate 4 terms of the fib sequence. This will keep the area smaller while producing two elements per cycle (a +2x increase over a CPU).

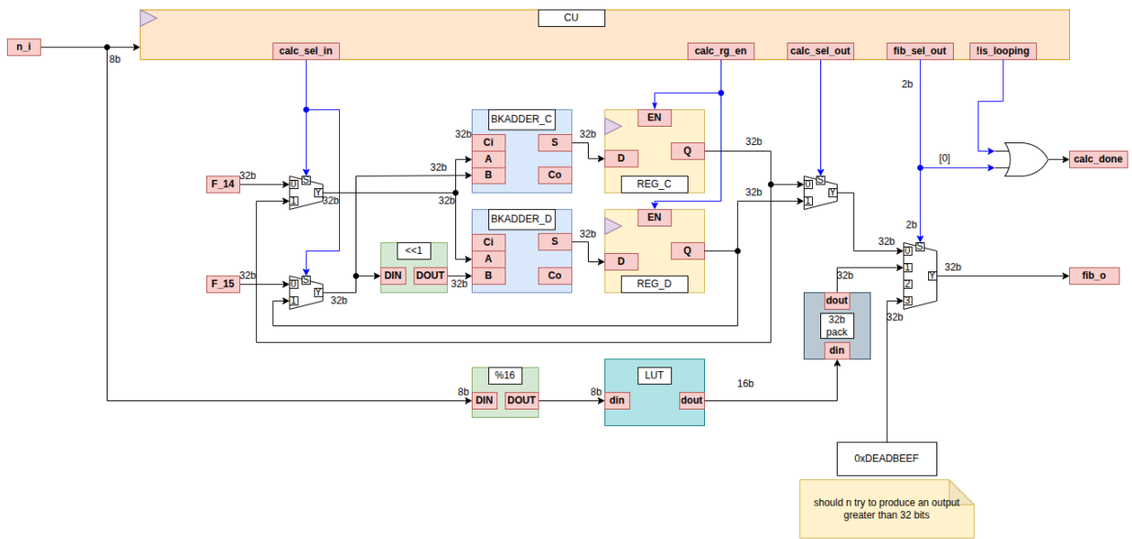


Figure 3.3: Computation Portion of Fib Accelerator

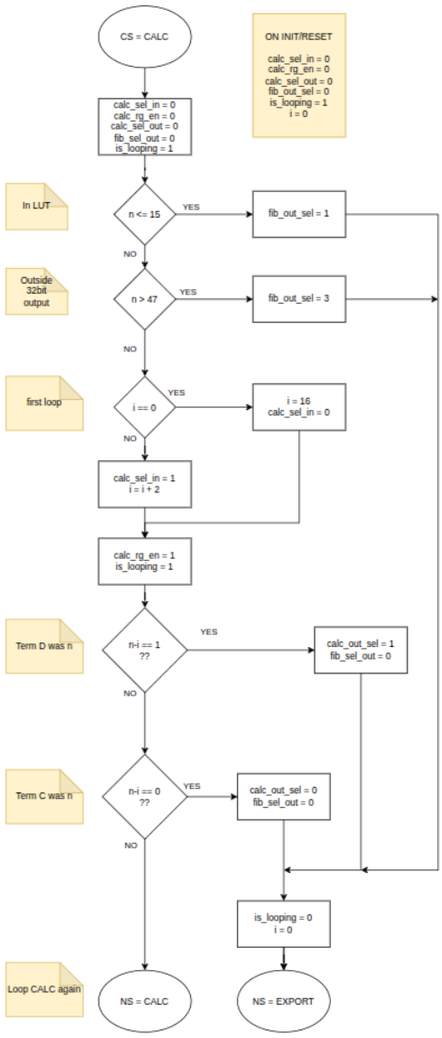
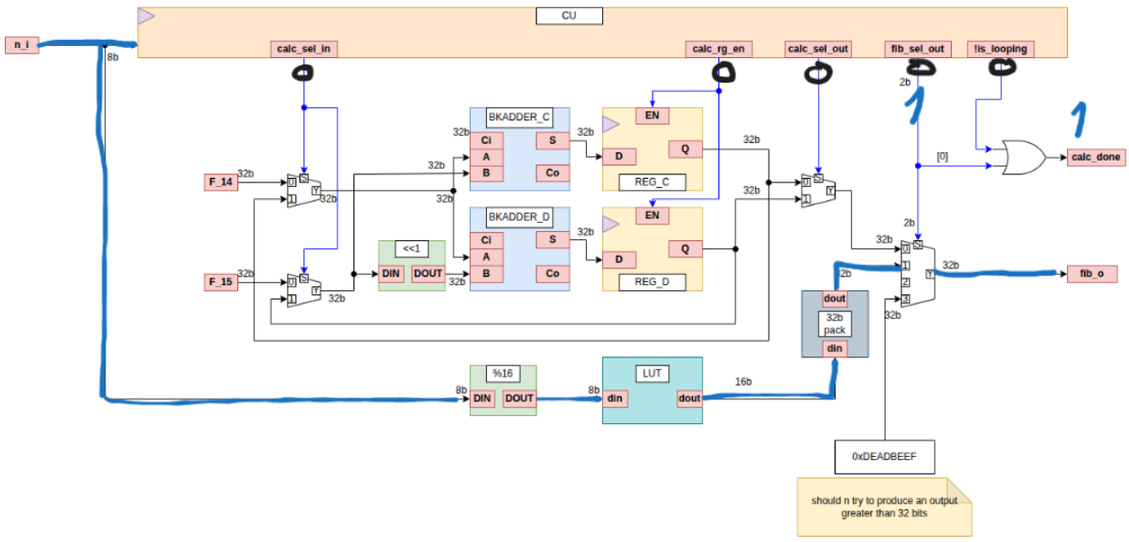


Figure 3.4: Control Flow of CALC State

To properly explain the design, let's trace through various values of n.



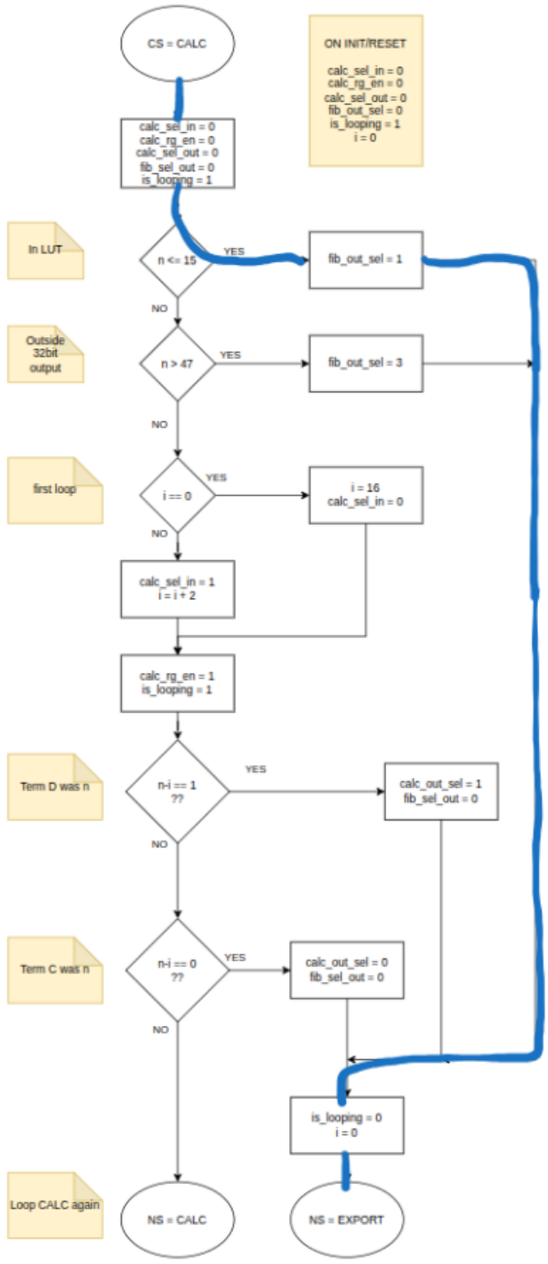
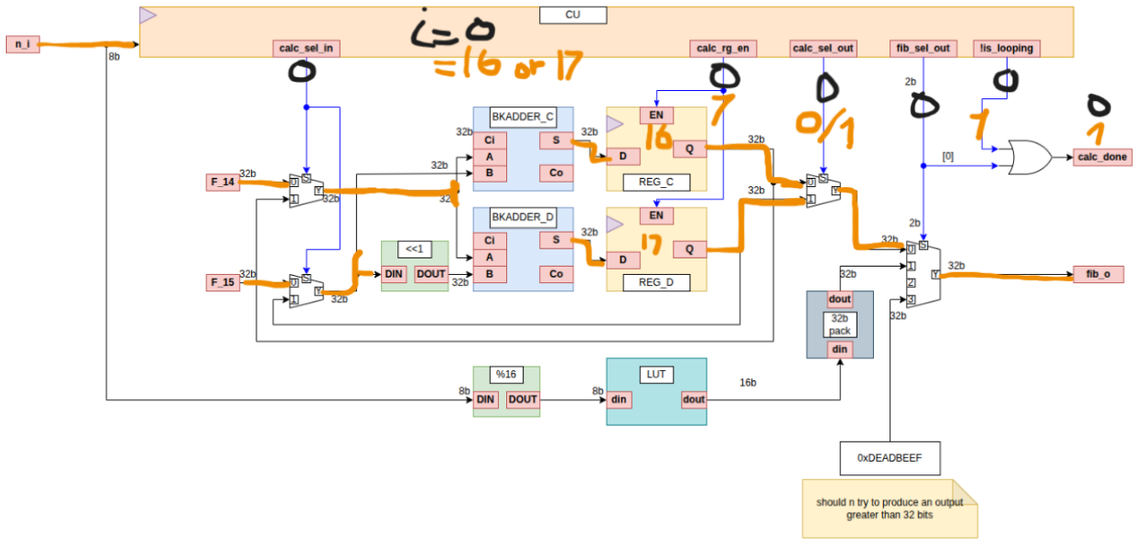


Figure 3.5: $n \leq 15$ uses the LUT



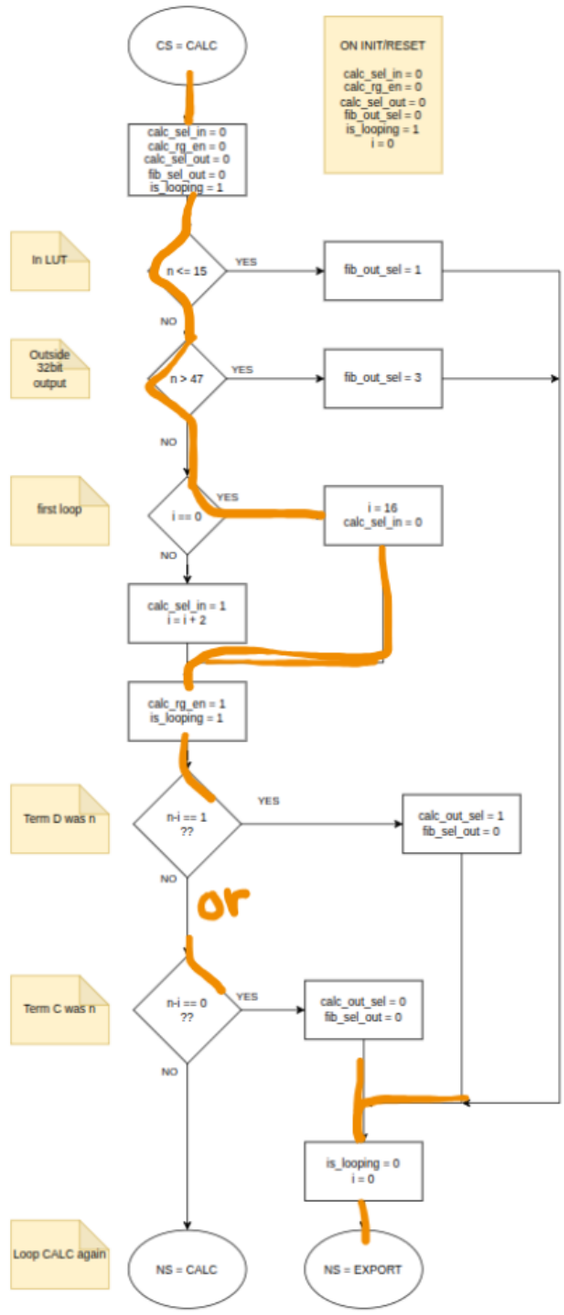


Figure 3.6: $n = 16$ or $n = 17$ calculates but doesn't loop

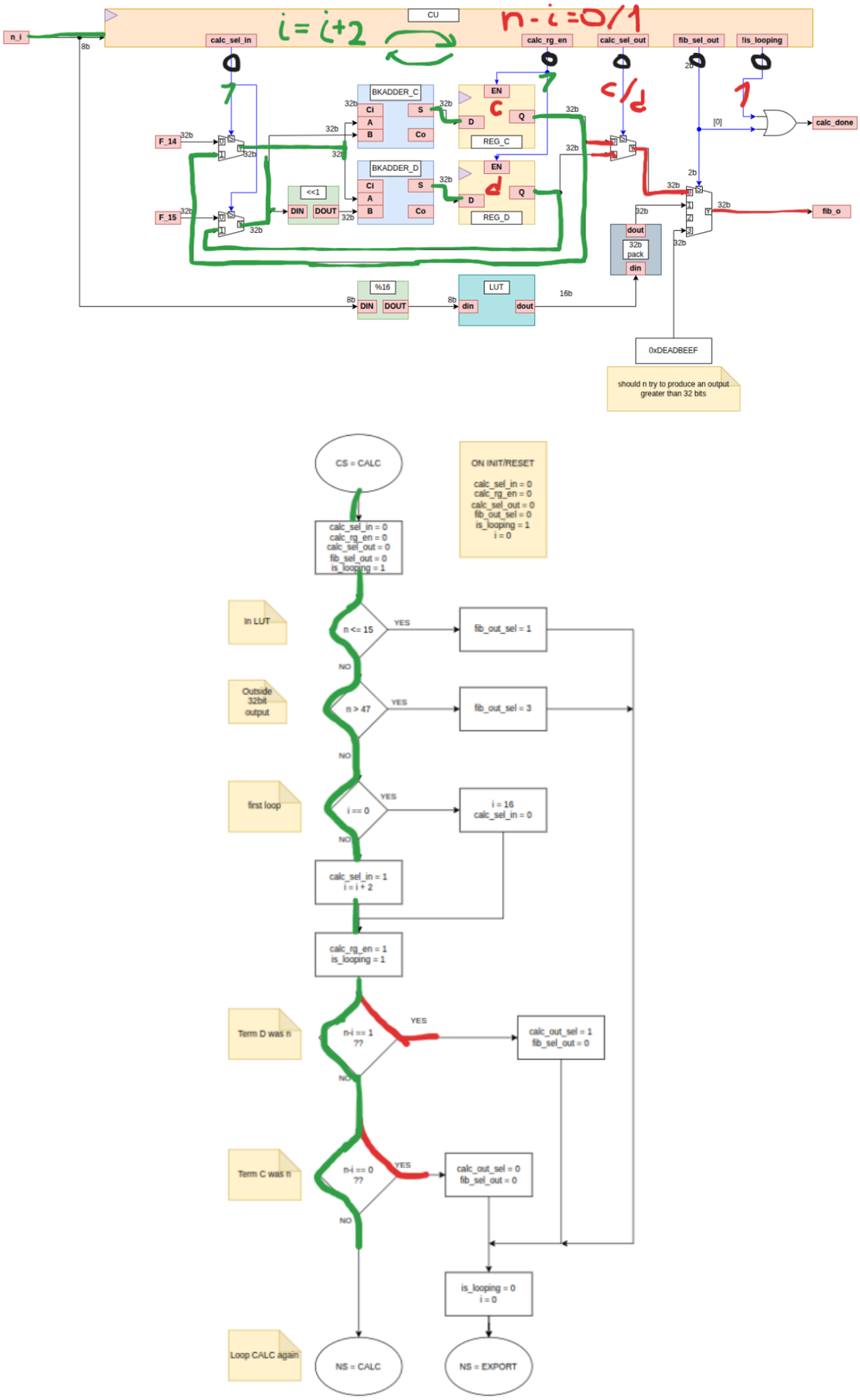


Figure 3.7: $n \geq 18$ and $n \leq 47$ calculates using loops

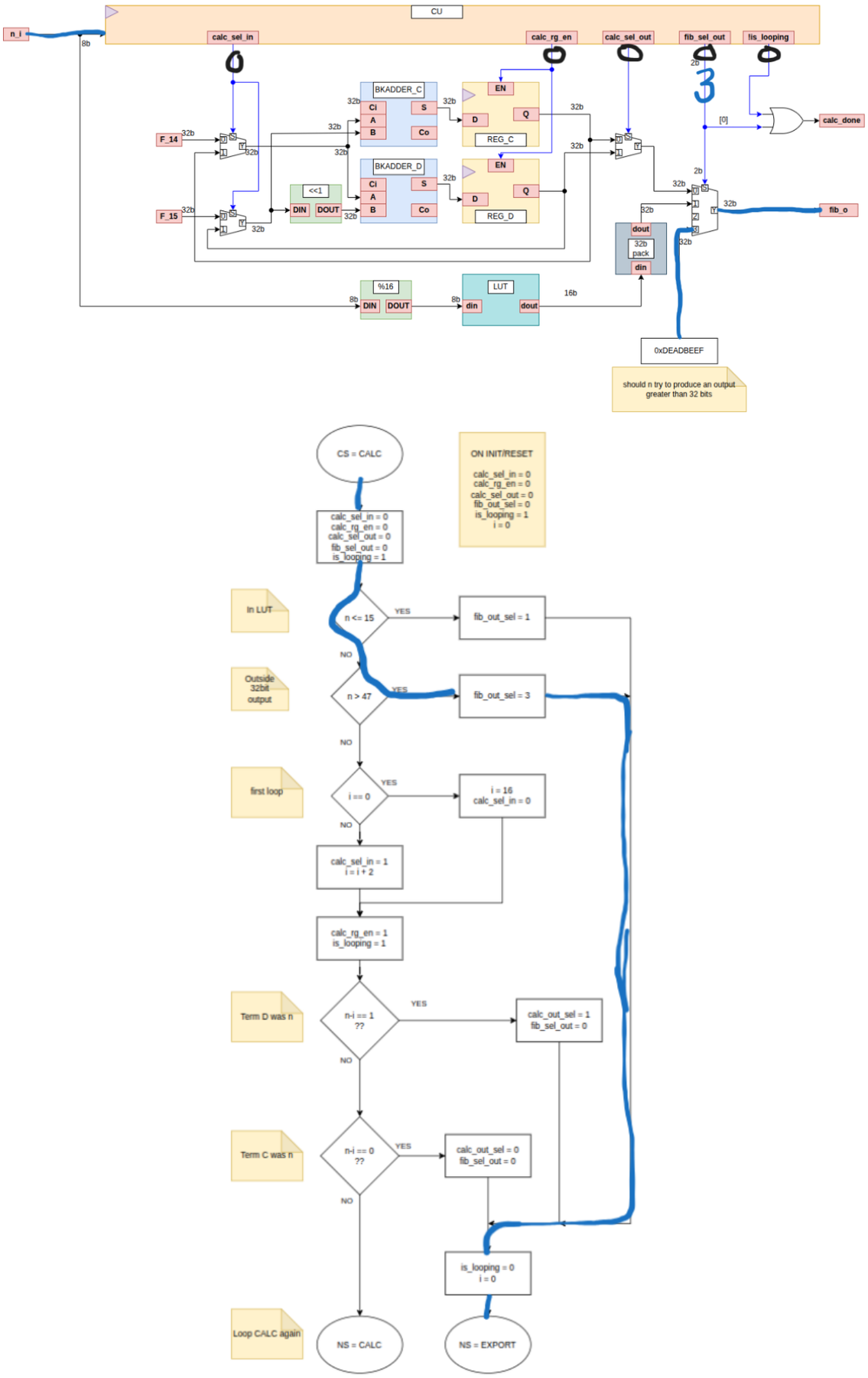


Figure 3.8: $n > 47$ defaults to avoid output overflow

3.3 Simulation Testing

3.3.1 Brent-Kung 'Parallel Prefix' Adder

The BKAdder was studied and implemented as a dynamic model, only verified using formal verification methods BMC and K-Induction. The rtl and verification can be seen in the same file 5.2.

Test Type	Result
BMC	✓
Induction	✓

Table 3.1: Verification Methods for BKAdder

3.3.2 Fibonacci Accelerator

The accelerator was initially verified using immediate and concurrent simulations on verilator (not icarus because SVA is not supported). The simulation file 5.4 tests:

- **Concurrents**
  - **reset_import**: confirms the module comes out of RESET, enters the IMPORT state, and signals the upstream that it is ready to accept an input n.
  - **import_export**: confirms the model is not holding the output steady while prompting the upstream for a new input n.
  - **hold_fib**: confirms the output `fib_o` was steady as long as our module is finished calculating and the downstream has not confirmed an output read.
- **Immediates**
  1. Test $n \in [0, 47]$ outputs the correct values sequentially.
  2. Test $n \in [48, 255]$ outputs an error code because output overflow is certain.

Test Type	Result
Verilator (SVA) Concurrents	✓
Verilator (SVA) Immediates	✓
Icarus (SVA) Immediates	✓
BMC	✗TODO
Induction	✗TODO

Table 3.2: Verification Methods for BKAdder

Chapter 4

Initial Post-Synthesis Analysis

Listing 4.1: Initial LibreLane config settings

```
---
DESIGN_NAME: fib
VERILOG_FILES: dir::rtl/*.sv
CLOCK_PERIOD: 20
CLOCK_PORT: clk_i
# Power Distribution Stuff
PDN_VOFFSET: 7
PDN_HOFFSET: 7
PDN_SKIPTRIM: true
RUN_POST_GRT_RESIZER_TIMING: true
RUN_POST_GRT_DESIGN_REPAIR: true

# Use our naming convention
SYNTH_AUTONAME: true
```

4.1 Static Timing Analysis (Pre-PlaceNRoute)

Listing 4.2: Overall Summary of Initial Post-Layout Simulation

```
Running 'Checker.SetupViolations' at 'runs/RUN_2026-10-29_15-38-45/74-checker-setupviolations'
Setup violations found in the following corners:
* max_ss_100C_1v60
* min_ss_100C_1v60
* nom_ss_100C_1v60
No setup violations found
Running 'Checker.HoldViolations' at 'runs/RUN_2026-10-29_15-38-45/75-checker-holdviolations'
No hold violations found
Running 'Checker.MaxSlewViolations' at 'runs/RUN_2026-10-29_15-38-45/76-checker-maxslewviolations'
Max Slew violations found in the following corners:
* max_ss_100C_1v60
* max_tt_025C_1v80
* min_ss_100C_1v60
* min_tt_025C_1v80
* nom_ss_100C_1v60
* nom_tt_025C_1v80
No max slew violations found
Running 'Checker.MaxCapViolations' at 'runs/RUN_2026-10-29_15-38-45/77-checker-maxcapviolations'
No max cap violations found
```

Listing 4.3: openroad-staprepnr Summary

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violatio.	Max Slew Violati.
Overall	0.0383	0.0383	0.0000	0	0	-11.7484	-11.7484	-614.05.	108	108	3	466
nom_tt_025C_1v80	0.1905	0.1905	0.0000	0	0	-1.5365	-1.5365	-32.1235	39	39	3	329
nom_ss_100C_1v60	0.6187	0.6187	0.0000	0	0	-11.7484	-11.7484	-614.05.	69	69	3	466
nom_ff_n40C_1v95	0.0383	0.0383	0.0000	0	0	3.0052	3.0052	0.0000	0	0	3	202

After performing an initial static-timing analysis, there is evidence of setup violations in the pre-place and route phase, with the biggest offending corner at `nom_ss_100C_1v60`. Since this corner is testing logic speeds at the lowest worse case, we can infer that our logic is too slow and that the longest and slowest paths cannot meet out target frequency of 10ns. Now that we've identified which corner is the biggest thorn, we can start to ask where and which path(s) are in violation.

NOTE: Lets reside in pre-pnr for now, since the likelihood our timing issues will resolve after placement is not good.

Listing 4.4: Route for Setup's Biggest Offender

```

===========================================================================
report_checks -path_delay max (Setup)
============================================================================
====================== nom_ss_100C_1v60 Corner ===================================

Startpoint: valid_o_sky130_fd_sc_hd__dfxtp_2_Q
            (rising edge-triggered flip-flop clocked by clk_i)
Endpoint: mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q
          (rising edge-triggered flip-flop clocked by clk_i)
Path Group: clk_i
Path Type: max

Fanout         Cap        Slew       Delay        Time   Description
---------------------------------------------------------------------------------------
                      0.150000    0.000000    0.000000   clock clk_i (rise edge)
                                  0.000000    0.000000   clock network delay (ideal)
                      0.150000    0.000000    0.000000 ^ valid_o_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
```

```
19      4    0.040429    0.187477    0.807686     0.807686 v valid_o_sky130_fd_sc_hd__dfxtp_2_Q/Q (sky130_fd_sc_hd__dfxtp_2)
20                                                           valid_o (net)
21                       0.187477    0.000000     0.807686 v CS[0]_sky130_fd_sc_hd__or2_2_B/A (sky130_fd_sc_hd__or2_2)
22      4    0.011350    0.159989    0.817762     1.625448 v CS[0]_sky130_fd_sc_hd__or2_2_B/X (sky130_fd_sc_hd__or2_2)
23                                                           CS[0]_sky130_fd_sc_hd__or2_2_B_X (net)
24                       0.159989    0.000000     1.625448 v fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
25    126    0.350231   12.214338    9.953817    11.579266 ^ fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
26                                                           fib_sel_out[0]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a32o_2_X_A2_sky130_fd_sc_hd__or2_2_X_B_sky130_fd_sc_
27                      12.214338    0.000000    11.579266 ^ mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
28      2    0.003792    0.265631    2.905223    14.484488 ^ mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
29                                                           mux_b.DIN[46]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
30                       0.265631    0.000000    14.484488 ^ mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
31      3    0.014004    0.112177    0.565484    15.049973 v mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
32                                                           mux_b.DIN[45]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_A2_sky130_fd_sc_hd__xnor2_2_Y_A (net)
33                       0.112177    0.000000    15.049973 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
34      1    0.001248    0.147204    1.126995    16.176968 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
35                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
36                       0.147204    0.000000    16.176968 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
37      2    0.003505    0.169459    1.044334    17.221302 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
38                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
39                       0.169459    0.000000    17.221302 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
40      3    0.007130    0.257365    1.464960    18.686262 v mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
41                                                           mux_b.DIN[47]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
42                       0.257365    0.000000    18.686262 v mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
43      5    0.013657    0.318167    0.429750    19.116013 ^ mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
44                                                           mux_b.DIN[51]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__a21oi_2_Y_A2_sky130_fd_sc_hd__xnor2_2_Y_A_sky130_fd_s
45                       0.318167    0.000000    19.116013 ^ mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
46      2    0.006094    0.215493    0.359973    19.475986 v mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
47                                                           mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
48                       0.215493    0.000000    19.475986 v mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
49      5    0.016287    0.363301    0.466146    19.942133 ^ mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
50                                                           mux_b.DIN[60]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o211a_2_X_B1_sky130_fd_sc_hd__o21ai_2_Y_A2_sky130_fd_
51                       0.363301    0.000000    19.942133 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
52      1    0.004617    0.070111    0.335543    20.277676 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B_sky130_fd_s
53                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y_B (net)
54                       0.070111    0.000000    20.277676 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y/B (sky130_fd_
55      1    0.002251    1.068041    0.090818    20.368494 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2_sky130_fd_sc_hd__nand2_2_Y/Y (sky130_fd_
56                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X_B2 (net)
57                       1.068041    0.000000    20.368494 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X/B2 (sky130_fd_sc_hd__o221a_2)
58      1    0.001577    0.082435    0.871987    21.240480 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D_sky130_fd_sc_hd__o221a_2_X/X (sky130_fd_sc_hd__o221a_2)
59                                                           mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q_D (net)
60                       0.082435    0.000000    21.240480 v mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q/D (sky130_fd_sc_hd__dfxtp_2)
61                                               21.240480   data arrival time
62
63                       0.150000   10.000000    10.000000   clock clk_i (rise edge)
64                                   0.000000    10.000000   clock network delay (ideal)
65                                  -0.250000     9.750000   clock uncertainty
66                                   0.000000     9.750000   clock reconvergence pessimism
67                                               9.750000 ^ mux_b.DIN[61]_sky130_fd_sc_hd__dfxtp_2_Q/CLK (sky130_fd_sc_hd__dfxtp_2)
68                                  -0.257883     9.492117   library setup time
69                                               9.492117   data required time
70 -----------------------------------------------------------------------------------------------
71                                               9.492117   data required time
72                                             -21.240480   data arrival time
73 -----------------------------------------------------------------------------------------------
74                                             -11.748363   slack (VIOLATED)
```

Looking at fig 4.4, there are a few items that stand out.

- L25: signal `fib_sel_out[0]` has a fanout of 126 when its rising ($\wedge$). Moreover, we occurred a massive delay and skew on this one transaction.
- L27-59: The end bits on the input mux b are DOWNSTREAM and ADDING to the delay of this path.

4.1.1 Analysis

- Q: Why is there such a large fanout on this signal? This signal is the SEL signal for the `mux_fib` in fig 3.3, meaning that it is connected to many smaller muxes to drive the 128-bit one. This, on the surface, doesn't sound too problematic if this was the only logic to be calculated. Might there be something else wrong?
- Lets scrutinize the path in fig 3.7 as I suspect its also causing unnecessary delay. First, lets outline what is actually happening here, assuming a newly calculated term is our requested one (b/c its worst case).

  1. While the CU flies through the conditional statements, the adders are beginning to calculate new terms for C and D.
  2. Register enable is high s.t. we capture these calculations.
  3. We configure **two** 32-bit muxes to deliver term D to `fib_o` as well as set `calc_done`.
  4. The adders finish and the registers capture.
  5. The output muxes receive the data on the next clock cycle.

  The initial inspiration was to store the new calculations in the registers as well as config the muxes, then by the EXPORT stage, we should hold and be ready to output the correct value. Unfortunately, this is too much logic to run through in a single stage and clock cycle. Moreover, a second 32-bit mux on the output is not needed.

4.1.2 Redesign

Chapter 5

Appendix

5.1 Code

5.1.1 RTL

Listing 5.1: Mux

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 04:24:06 PM
// Design Name:
// Module Name: Mux_N
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//                          din[0] occupies the LSB WIDTH slot
//////////////////////////////////////////////////////////////////////////////////


module Mux_N
#(
    parameter WIDTH     = 32,
    parameter INPUTS    = 4,
    parameter SEL_WIDTH = (INPUTS <= 1) ? 1 : $clog2(INPUTS)
)
(
    input logic     [INPUTS-1:0][WIDTH-1:0] DIN,
    input logic     [SEL_WIDTH-1:0]         SEL,
    output logic    [WIDTH-1:0]             DOUT
);

    always_comb begin

        // ~~ default ~~
        DOUT = '0;

        if(int'(SEL) < INPUTS) begin
            DOUT = DIN[SEL];
        end
    end

endmodule
```

Listing 5.2: Brent-Kung 'Parallel Prefix' Adder

```
`timescale 1ns / 1ps

module BKAdder
#( parameter int WIDTH = 32 )
(
    input  logic [WIDTH-1:0] a,
    input  logic [WIDTH-1:0] b,
    input  logic             cin,
    output logic [WIDTH-1:0] sum,
    output logic             cout
);

    // ~~~~ local vars ~~~~
    // Keep at least one tree level so WIDTH=1 is supported.
    localparam int LEVELS = (WIDTH <= 1) ? 1 : $clog2(WIDTH);
    localparam int N      = 1 << LEVELS;
    localparam int STAGES = 2*LEVELS - 1;

    wire [N-1:0] a_pad = {{(N-WIDTH){1'b0}}, a};
    wire [N-1:0] b_pad = {{(N-WIDTH){1'b0}}, b};

    // Prefix arrays: stage 0 contains each bit's individual terms.
    wire [N-1:0] P [0:STAGES];
    wire [N-1:0] G [0:STAGES];

    assign P[0] = a_pad ^ b_pad;
    assign G[0] = a_pad & b_pad;
```

```
28
29      // ~~~~ Up-sweep: combine adjacent groups ~~~~
30      for (genvar s = 0; s < LEVELS; s++) begin : gen_up
31          localparam int SPAN = 1 << s;
32          localparam int PREV = s;
33          localparam int NEXT = s + 1;
34
35          for (genvar i = 0; i < N; i++) begin : gen_bit
36              if (((i + 1) % (2*SPAN)) == 0) begin : gen_combine
37                  assign G[NEXT][i] =
38                      G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
39                  assign P[NEXT][i] =
40                      P[PREV][i] & P[PREV][i-SPAN];
41              end else begin : gen_pass
42                  assign G[NEXT][i] = G[PREV][i];
43                  assign P[NEXT][i] = P[PREV][i];
44              end
45          end
46      end
47
48      // ~~~~ Down-sweep: fill in the remaining prefixes ~~~~
49      for (genvar t = 0; t < LEVELS-1; t++) begin : gen_down
50          localparam int D    = LEVELS - 2 - t;
51          localparam int SPAN = 1 << D;
52          localparam int PREV = LEVELS + t;
53          localparam int NEXT = PREV + 1;
54
55          for (genvar i = 0; i < N; i++) begin : gen_bit
56              if ((i >= 3*SPAN - 1) &&
57                  (((i - (3*SPAN - 1)) % (2*SPAN)) == 0)) begin : gen_combine
58                  assign G[NEXT][i] =
59                      G[PREV][i] | (P[PREV][i] & G[PREV][i-SPAN]);
60                  assign P[NEXT][i] =
61                      P[PREV][i] & P[PREV][i-SPAN];
62              end else begin : gen_pass
63                  assign G[NEXT][i] = G[PREV][i];
64                  assign P[NEXT][i] = P[PREV][i];
65              end
66          end
67      end
68
69      wire [N-1:0] carry;
70
71      // ~~~~ collect sum and carry out bit ~~~~
72      for (genvar k = 0; k < WIDTH; k++) begin : gen_sum
73          if (k == 0) begin : gen_first
74              assign carry[k] = cin;
75          end else begin : gen_other
76              assign carry[k] =
77                  G[STAGES][k-1] | (P[STAGES][k-1] & cin);
78          end
79
80          assign sum[k] = P[0][k] ^ carry[k];
81      end
82
83      assign cout =
84          G[STAGES][WIDTH-1] | (P[STAGES][WIDTH-1] & cin);
85
86
87      `ifdef FORMAL
88          always @(*) begin
89              assert( (a + b + cin) == {cout, sum});
90          end
91      `endif
92
93  endmodule
```

Listing 5.3: Fibonacci Accelerator

```
1   `timescale 1ns / 1ps
2
3   /*
4   * Purpose : creating a fibonacci seq
5   *
6   * Inputs :
7   *       clk_i       : clock w/ pos active edge
8   *       rstn_i      : active low synch reset (resets on zero)
9   *       n_i         : nth fib sequence to stop at
10  *       valid_i     : driven high when n_i is ready to be read
11  *       rdy_out_i   : driven high when module's output has been read
12  *
13  * Returns :
14  *       rdy_in_o    : driven high by module to request new input data
15  *       fib_o       : calculated Fibonacci[n]
16  *       valid_o     : driven high by module when output data is ready
17  */
18
19  module fib
20  #(
21      parameter INBITS    = 8,
22      parameter OUTBITS   = 32
23  )
24  (
25      input logic                 clk_i,
26      input logic                 rstn_i,
27      input logic [INBITS-1:0]    n_i,
28      input logic                 valid_i,
29      input logic                 rdy_out_i,
30
31      output logic                rdy_in_o,
32      output logic [OUTBITS-1:0]  fib_o,
33      output logic                valid_o
34  );
35
36
37      // ~~~~ local vars ~~~~
38      typedef enum logic [2:0] {RESET, IMPORT, PRECALC, CALC, EXPORT} e_state;
```

```
    e_state CS, NS;

    localparam logic [7:0] LUTMAX       = 8'h0F; //15
    localparam logic [7:0] NMAX         = 8'h2F; //47
    localparam logic [31:0] INACTIVE    = 32'hFFFF_FFFF;

    function logic [15:0] LUT (input logic [3:0] idx);
        case(idx)
            4'd0:       LUT = 16'd0;
            4'd1:       LUT = 16'd1;
            4'd2:       LUT = 16'd1;
            4'd3:       LUT = 16'd2;
            4'd4:       LUT = 16'd3;
            4'd5:       LUT = 16'd5;
            4'd6:       LUT = 16'd8;
            4'd7:       LUT = 16'd13;
            4'd8:       LUT = 16'd21;
            4'd9:       LUT = 16'd34;
            4'd10:      LUT = 16'd55;
            4'd11:      LUT = 16'd89;
            4'd12:      LUT = 16'd144;
            4'd13:      LUT = 16'd233;
            4'd14:      LUT = 16'd377;
            4'd15:      LUT = 16'd610;
            default:    LUT = 16'hDEAD;
        endcase
    endfunction

    logic               sel_in,
                        rg_en;
    logic [1:0]         sel_out_next,
                        sel_out_curr;
    logic [OUTBITS-1:0] regC, regD;
    logic [INBITS-1:0]  n_curr,
                        n_next,
                        i,
                        i_next;


    // ~~~~ module instantiations ~~~~

    logic [1:0][OUTBITS-1:0]mux_din_a,
                            mux_din_b;
    logic [3:0][OUTBITS-1:0]mux_din_c;
    logic [OUTBITS-1:0]     mux_dout_a,
                            mux_dout_b;

    assign mux_din_a    = {regC, {16'd0, 16'd377} };
    assign mux_din_b    = {regD, {16'd0, 16'd610} };
    assign mux_din_c    = {
                            regC,
                            regD,
                            {16'd0, LUT(n_curr[3:0])},
                            INACTIVE
                        };

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_a
    (
        .DIN        (mux_din_a),
        .SEL        (sel_in),
        .DOUT       (mux_dout_a)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (2),
        .SEL_WIDTH  ()
    )
    mux_b
    (
        .DIN        (mux_din_b),
        .SEL        (sel_in),
        .DOUT       (mux_dout_b)
    );

    Mux_N
    #(
        .WIDTH      (OUTBITS),
        .INPUTS     (4),
        .SEL_WIDTH  ()
    )
    mux_fib
    (
        .DIN        (mux_din_c),
        .SEL        (sel_out_curr),
        .DOUT       (fib_o)
    );

    logic [OUTBITS-1:0] c_sum, d_sum;
    logic               unused1, unused2;

    BKAdder
    #( .WIDTH(OUTBITS))
    AddrC
    (
        .a(mux_dout_a),
        .b(mux_dout_b),
        .cin(1'b0),
        .sum(c_sum),
        .cout(unused1)
    );
```

```
147
148     BKAdder
149     #( .WIDTH(OUTBITS))
150     AddrD
151     (
152         .a(mux_dout_a),
153         .b(mux_dout_b << 1),
154         .cin(1'b0),
155         .sum(d_sum),
156         .cout(unused2)
157     );
158
159
160     // ~~~~ synch logic ~~~~
161     always_ff @(posedge clk_i) begin
162
163         // ~~ priority 0 : reset
164         if(rstn_i == 1'b0 ) begin
165             i               <= '0;
166             n_curr          <= '0;
167             sel_out_curr    <= 2'b00;
168             regC            <= '0;
169             regD            <= '0;
170             CS              <= RESET;
171         end
172
173         // ~~ priority 1 : state wiggles ~~
174         else begin
175             i               <= i_next;
176             n_curr          <= n_next;
177             sel_out_curr    <= sel_out_next;
178
179             if(rg_en) begin
180                 regC        <= c_sum;
181                 regD        <= d_sum;
182             end
183
184             CS              <= NS;
185         end
186     end
187
188
189
190     // ~~~~ comb logic ~~~~
191     always_comb begin
192
193         // ~~ system defaults ~~
194         rdy_in_o        = 1'b0;
195         valid_o         = 1'b0;
196         i_next          = '0;
197         n_next          = n_curr;
198
199         // ~ CALC defaults ~
200         sel_in          = 1'b0;
201         rg_en           = 1'b0;
202         sel_out_next    = sel_out_curr;
203
204         case(CS)
205
206             RESET:
207             begin
208                 sel_out_next    = '0;
209                 n_next          = '0;
210                 NS              = IMPORT;
211             end
212
213             IMPORT:
214             begin
215                 sel_out_next    = '0;
216                 rdy_in_o        = 1'b1;
217
218                 if(valid_i) begin
219                     n_next      = n_i;
220                     NS          = PRECALC;
221                 end
222                 else begin
223                     n_next      = '0;
224                     NS          = IMPORT;
225                 end
226             end
227
228             PRECALC:
229             begin
230
231                 rg_en           = 1'b1;     // store f16 and f17
232
233                 // Priority 0: n is inside LUT
234                 if(n_curr <= LUTMAX) begin
235                     sel_out_next    = 2'd1;
236                     NS              = EXPORT;
237                 end
238
239                 // Priority 1: (edge case) fib will be greater than OUTBITS
240                 else if(n_curr > NMAX) begin
241                     sel_out_next    = 2'd0;
242                     NS              = EXPORT;
243                 end
244                 // Priority 2: n is between [16,47]
245                 else begin
246                     sel_out_next    = 2'd0;     // default to inactive
247                     i_next          = 8'd16;    // init counting term
248                     NS              = CALC;
249                 end
250
251             end
252
253             CALC:
254             begin
```

```
255
256                 sel_in  = 1'b1; // use prev calc
257
258                 // Priority 0: found term C
259                 if( (n_curr - i) == 0 ) begin
260                     sel_out_next    = 2'd3;
261                     NS              = EXPORT;
262                 end
263                 // Priority 1: found term D
264                 else if((n_curr - i) == 1 ) begin
265                     sel_out_next    = 2'd2;
266                     NS              = EXPORT;
267                 end
268                 // Priority 2: keep counting
269                 else begin
270                     rg_en           = 1'b1;     // capture current adder calc
271                     i_next          = i + 8'd2;
272                     NS              = CALC;
273                 end
274
275             end
276
277             EXPORT:
278             begin
279                 valid_o     = 1'b1;
280                 if(rdy_out_i == 1'b1)   NS = IMPORT;
281                 else                    NS = EXPORT;
282             end
283
284             default:
285             begin
286                 NS          = RESET;
287             end
288         endcase
289     end
290
291     `ifdef FORMAL
292
293         // ======================
294         // locals
295         // ======================
296         localparam logic [31:0] FIBMAX      = 32'd2971215073;
297         logic                   f_past_valid;
298         initial                 f_past_valid = 0;
299         always @(posedge clk_i) f_past_valid <= 1;
300         always_comb             if(!f_past_valid) assume (!rstn_i);
301
302         logic   f_waiting,
303                 f_reading,
304                 f_calcing,
305                 f_writing;
306
307         assign  f_waiting = rdy_in_o && !valid_i;
308         assign  f_reading = rdy_in_o && valid_i;
309         assign  f_calcing = !rdy_in_o && !valid_o;
310         assign  f_writing = !rdy_in_o && valid_o;
311
312         // ======================
313         // Fib Boundary Checks (TODO not passing Ind)
314         // ======================
315
316         // a zero output must mean the input was zero (not including reset)
317         always @(posedge clk_i) begin
318             if( (fib_o == 32'd0 && $rose(f_writing)) &&
319                 (rstn_i && $past(rstn_i) && $past(rstn_i, 2) && $past(rstn_i, 3)) &&
320                 (f_past_valid && $past(f_past_valid) && $past(f_past_valid, 2) && $past(f_past_valid, 3))
321             ) begin
322                 assert($past(n_i, 2) == 8'd0);
323                 assert($past(f_reading, 2));
324             end
325         end
326
327
328         // high values of n should take 2 cycles to output the ceil code
329         always @(posedge clk_i) begin
330             if(f_past_valid && (fib_o == INACTIVE && $rose(f_writing))) begin
331                 assert($past(n_i,2) > NMAX);
332                 assert($past(f_reading, 2));
333             end
334         end
335
336
337         // ======================
338         // Data Integrity (TODO not passing Ind)
339         // ======================
340         function automatic longint unsigned fibboni(
341             input logic [INBITS-1:0] n
342         );
343             longint unsigned a, b, t;
344             a       = 0;
345             b       = 1;
346             for(int i = 0; i < NMAX; i++) begin
347                 if(i < n) begin
348                     t = a + b;
349                     a = b;
350                     b = t;
351                 end
352             end
353             fibboni = a;
354         endfunction
355
356     `endif
357
358 endmodule
```

5.1.2 Simulation SVA Testing

Listing 5.4: Testbench simulation for Fibonacci Accelerator

```
`timescale 1ns / 1ps

module tb_fib();

    // ~~~~ imports ~~~~
    import tb_utils_pkg::*;

    // ~~~~ locals ~~~~
    localparam      CLK_PERIOD = 10;
    logic           clk,
                    resetn,
                    n_is_valid,
                    fib_was_read,
                    rdy_new_n,
                    fib_is_valid;

    logic [7:0]     n;
    logic [31:0]    fib_actual, fib_expected;
    localparam  logic [31:0]ERR_CODE    = 32'hFFFF_FFFF;
    localparam  logic [7:0] INBITS      = 8;
    localparam  logic [7:0] NMAX        = 8'd47;

    // ~~~~ functions ~~~~
    function automatic longint unsigned fibboni(
        input logic [INBITS-1:0] N
    );
        longint unsigned a, b, t;
        a       = 0;
        b       = 1;
        for(int unsigned i = 0; 8'(i) < NMAX; i++) begin
            if(i < N) begin
                t = a + b;
                a = b;
                b = t;
            end
        end
        fibboni = a;
    endfunction

    // ~~~~ tasks ~~~~
    task automatic request_fn(
        input logic [7:0]   n_val
    );
        // ~ IMPORT ~
        do @(posedge clk);
        while(rdy_new_n !== 1'b1);

        n               = n_val;
        n_is_valid      = 1'b1;
        @(posedge clk);

        // ~ CALC -> EXPORT ~
        do @(posedge clk);
        while(fib_is_valid !== 1'b1);
        n_is_valid      = 1'b0;
        @(posedge clk);
    endtask

    // ~~~~ instantiations ~~~~
    fib
    #(
        .INBITS     (8),
        .OUTBITS    (32)
    )
    Fib
    (
        .clk_i      (clk),
        .rstn_i     (resetn),
        .n_i        (n),
        .valid_i    (n_is_valid),
        .rdy_out_i  (fib_was_read),

        .rdy_in_o   (rdy_new_n),
        .fib_o      (fib_actual),
        .valid_o    (fib_is_valid)
    );

    // ~~~~ heartbeat ~~~~
    always #(CLK_PERIOD/2) clk = ~clk;


    // ~~~~ testing ~~~~
    testcase tc;
    logic [31:0] tnum;

    `ifndef NO_SVA
        // ~~ sequences (unsupported by icarus) ~~

        // ~~ properties (unsupported by icarus) ~~
        property p1;
            @(posedge clk) disable iff(!resetn)
                $rose(resetn) |-> rdy_new_n;
        endproperty

        property p2;
            @(posedge clk) disable iff(!resetn)
                not(rdy_new_n && fib_is_valid);
        endproperty

        property p3;
            @(posedge clk) disable iff(!resetn)
                ($past(fib_is_valid) && fib_is_valid && !fib_was_read) |->
                    ($past(fib_actual) == fib_actual);
        endproperty

        // ~~ concurrents (unsupported by icarus)~~
```

```
        reset_import    : assert property(p1) else tc.err("Didn't enter IMPORT after RESET");
        import_export   : assert property(p2) else tc.err("Shouldn't accept new data while holding old");
        hold_fib        : assert property(p3) else tc.err("fib output wasn't held");
    `endif

    // ~~~~ time-dependent testing ~~~~
    initial begin

        // ~~ waveforms ~~
        $dumpfile("tb_fib.vcd");
        $dumpvars(0);

        // ~~ defaults ~~
        tc              = new();
        clk             = 1'b1;
        resetn          = 1'b0;
        n               = 8'h00;
        n_is_valid      = 1'b0;
        fib_was_read    = 1'b0;

        // ~~ fresh start ~~
        #100;
        resetn          = 1'b1;

        // ~~ immediate ~~
        tc.new_test("n in [0,47]");
        tnum = tc.get_testnum();

            for(logic [7:0] i = 8'h00; i < 8'd48; i = i + 1'b1) begin

                request_fn(.n_val(i));
                fib_expected = 32'(fibboni(n));
                assert(fib_expected === fib_actual) else begin tc.err("fib value didnt match");end

                repeat(2)begin @(posedge clk); end
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
            end
            @(posedge clk);
        tc.test_done;

        tc.new_test("n in 48+");
        tnum = tc.get_testnum();

            for(logic [7:0] i = 8'd48; i < 8'hFF; i = i + 1'b1) begin

                request_fn(.n_val(i));
                assert(fib_actual == ERR_CODE) else begin tc.err("fib value didnt match"); end

                repeat(2)begin @(posedge clk); end
                fib_was_read    = 1'b1;
                @(posedge clk);
                fib_was_read    = 1'b0;
            @(posedge clk);
            end
        tc.test_done;

        $finish();
    end

endmodule
```

5.2 Boneyard

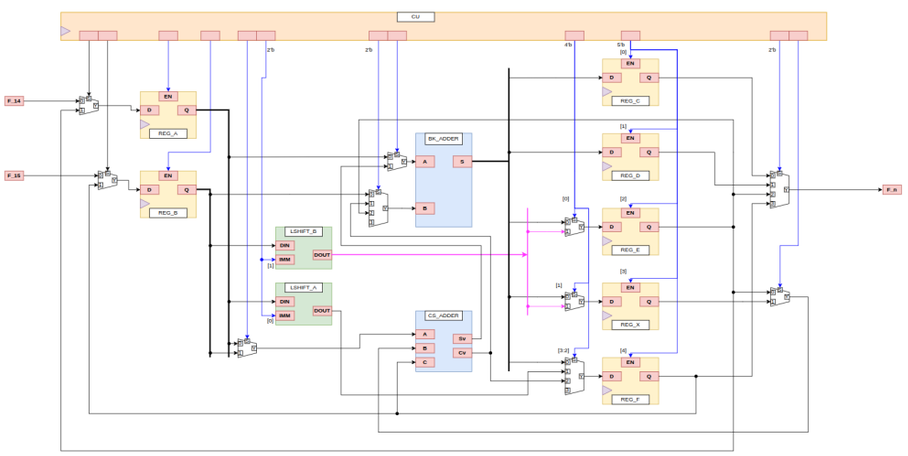


Figure 5.1: First Draft of Accelerator

Chapter 6

References

1. CPE490 - OpenSource ASIC